INVOICE – SIGNING PROCESS

Step by Step Towards Acquiring a Signed Invoice

فـاتورة
Fatoora

هيئة الزكاة والضريبة والجمارك
Zakat, Tax and Customs Authority

SHA-256 Hash

Hashing algorithm

Why

The main reason for using SHA-256 is to strengthen the security and protecting the data knowing that it doesn’t have any known vulnerabilities that make it insecure, and it has not been “broken” unlike some other popular hashing algorithms.

Output

the output of a hashing algorithm SHA256 will always be the same. 256 bits, which is 32 bytes, which is displayed as 64 alphanumeric characters.

Generate Invoice Hash

1st step in the signing process

Tags to be removed from invoice	XPath, Use this path to find the target tag
UBLExtension	*[local-name()='Invoice']//*[local-name()='UBLExtensions']
QR	//*[local-name()='AdditionalDocumentReference'][cbc:ID[normalize-space(text()) = 'QR']]
Signature	*[local-name()='Invoice']//*[local-name()='Signature']

What To Do

1. Open the invoice XML file.
2. Remove the tags mentioned in the table above using the XPath.
3. Remove the XML version.
4. Canonicalize the Invoice using the C14N11 standard
5. Hash the new invoice body using SHA-256 (**output**).
   e.g.:a11b6fe587a50f7daffe3a7fb42dcccf32b43ee9b37d9f252d04243e54c11a3f
6. Encode the hashed invoice using base64 (**output**)
   - Using HEX-to Base64 Encoder

   e.g.:oRtv5YelD32v/jp/tC3MzzK0PumzfZ8lLQQkPlTBGj8=

Note:
-All these values will be used in later steps.
-Please make sure that you have a copy of the original invoice before removing the above tags

Generate Digital Signature
2nd step in the signing process

Values to be used
Generated Invoice Hash from 1st step (in SHA-256 format **not** encoded with base64)
Private key

What To Do

1. Generate private key from CSR config file (you can refer to openssl commands, or readme file on SDK)
2. Sign the generated invoice hash (in SHA-256 format **not** encoded with base64) with ECDSA using the private key (**output**).
   e.g.:MEQCIGvLa1f3uMCe0AidKUWJ5ghMiDMRcC0qO78ntcTKVOYgAiAKBkX+uuFhbIcye3JznNa45qH1twlLFu/qPzEQ9HMNLw==

Note: This value will be used in later steps.

Generate Certificate Hash

3rd step in the signing process

Values to be used
X509 Certificate(After completing CCSID API, it will return (binary security token), take this value and decode it using base 64, the output is X509 certificate.)

What To Do

1. Hash the x509 certificate using SHA-256 (**output**).
   e.g.:69a95fc237b42714dc4457a33b94cc452fd9f110504c683c401144d9544894fb
2. Encode the hashed x509 certificate using base64 (**ENCODER BASE64**) (**output**).
   e.g.:NjlhOTVmYzIzN2I0MjcxNGRjNDQ1N2EzM2I5NGNjNDUyZmQ5ZjExMDUwNGM2ODNjNDAxMTQ0ZDk1NDQ4OTRmYg==

Note: final output will be used in later steps

Populate The Signed Properties Output

4th step in the signing process

Fields	Values	XPath
DigestValue	Final output from 3rd step	/Invoice/ext:UBLExtensions/ext:UBLExtension/ext:ExtensionContent/sig:UBLDocumentSignatures/sac:SignatureInformation/ds:Signature/ds:Object/xades:QualifyingProperties/xades:**SignedProperties**/xades:SignedSignatureProperties/xades:SigningCertificate/xades:Cert/xades:CertDigest/ds:DigestValue
SigningTime	Sign timestamp as current datetime(current dateTime)	/Invoice/ext:UBLExtensions/ext:UBLExtension/ext:ExtensionContent/sig:UBLDocumentSignatures/sac:SignatureInformation/ds:Signature/ds:Object/xades:QualifyingProperties/xades:**SignedProperties**/xades:SignedSignatureProperties/xades:SigningTime
X509IssuerName	Certificate issuer name From the X509 certificate	/Invoice/ext:UBLExtensions/ext:UBLExtension/ext:ExtensionContent/sig:UBLDocumentSignatures/sac:SignatureInformation/ds:Signature/ds:Object/xades:QualifyingProperties/xades:**SignedProperties**/xades:SignedSignatureProperties/xades:SigningCertificate/xades:Cert/xades:IssuerSerial/ds:X509IssuerName
X509SerialNumber	Certificate serial number From the X509 certificate	/Invoice/ext:UBLExtensions/ext:UBLExtension/ext:ExtensionContent/sig:UBLDocumentSignatures/sac:SignatureInformation/ds:Signature/ds:Object/xades:QualifyingProperties/xades:**SignedProperties**//xades:SignedSignatureProperties/xades:SigningCertificate/xades:Cert/xades:IssuerSerial/ds:X509SerialNumber

What To Do

1. **Open the invoice before 1st step (before getting tags removed)**
2. Refer to the above table to fill mentioned fields with their corresponding values using the related Xpath, (if there are any old values already exist in the fields, please make sure to remove all of them and replace them with the new values only).
3. To get X509 Serial number, decode the X509 certificate the value will be printed in the decoded result.

Notes:

- Populated **Signed Properties** will be used in the next step.

Signed Properties tag.

You should use this tag in the next step.

```
<xades:SignedProperties xmlns:xades="http://uri.etsi.org/01903/v1.3.2#" Id="xadesSignedProperties">
    <xades:SignedSignatureProperties>
      <xades:SigningTime></xades:SigningTime>
      <xades:SigningCertificate>
        <xades:Cert>
          <xades:CertDigest>
            <ds:DigestMethod xmlns:ds="http://www.w3.org/2000/09/xmldsig#" Algorithm="http://www.w3.org/2001/04/xmlenc#sha256"/>
            <ds:DigestValue xmlns:ds="http://www.w3.org/2000/09/xmldsig#"></ds:DigestValue>
          </xades:CertDigest>
          <xades:IssuerSerial>
            <ds:X509IssuerName xmlns:ds="http://www.w3.org/2000/09/xmldsig#"></ds:X509IssuerName>
            <ds:X509SerialNumber xmlns:ds="http://www.w3.org/2000/09/xmldsig#"></ds:X509SerialNumber>
          </xades:IssuerSerial>
        </xades:Cert>
      </xades:SigningCertificate>
    </xades:SignedSignatureProperties>
  </xades:SignedProperties>
```

Note:

-You shouldn’t include this tag in the invoice, we just using it to populate Signed Properties Hash.

Generate Signed Properties Hash

5th step in the signing process

What To Do

1. To generate the Signed Properties Hash, you should use the tag provided on the previous page and fill in the Populated Signed Properties in step 4 (using the same values).
2. Hash the new property tag(After fill) using SHA-256 (**output**).
3. **e.g.:99282555b5d79209be5883cc23eb234cd01bd33ea7d54d88f491248d33e321f1**
4. Encode the hashed property using base64 (**ENCODER BASE64**) (**output**).
   E.g.:OTkyODI1NTViNWQ3OTIwOWJlNTg4M2NjMjNlYjIzNGNkMDFiZDMzZWE3ZDU0ZDg4ZjQ5MTI0OGQzM2UzMjFmMQ==

Note:
-The final output will be used in later steps .

Populate The UBL Extensions Output

6th step in the signing process

Fields	Values	XPath
SignatureValue	Digital Signature from the 2nd Step	/Invoice/ext:UBLExtensions/ext:UBLExtension/ext:ExtensionContent/sig:UBLDocumentSignatures/sac:SignatureInformation/ds:Signature/ds:SignatureValue
X509Certificate	Certificate	/Invoice/ext:UBLExtensions/ext:UBLExtension/ext:ExtensionContent/sig:UBLDocumentSignatures/sac:SignatureInformation/ds:Signature/ds:KeyInfo/ds:X509Data/ds:X509Certificate
DigestValue	Encoded signed Properties hash from the 5th Step	/Invoice/ext:UBLExtensions/ext:UBLExtension/ext:ExtensionContent/sig:UBLDocumentSignatures/sac:SignatureInformation/ds:Signature/ds:SignedInfo/ds:Reference[@URI='#xadesSignedProperties']/ds:DigestValue
DigestValue	Encoded invoice hash from the 1st Step	/Invoice/ext:UBLExtensions/ext:UBLExtension/ext:ExtensionContent/sig:UBLDocumentSignatures/sac:SignatureInformation/ds:Signature/ds:SignedInfo/ds:Reference[@Id='invoiceSignedData']/ds:DigestValue

What To Do

1. Use the invoice XML file acquired after completing the 4th step.
2. Refer to the above table to fill mentioned **UBL-Extensions tag's** fields with their corresponding values using the related XPath.

Note: if there are any old values already exist in the fields, please make sure to remove all of them and replace them with the new values only.

Generate QR & Populate Encoded QR

Final step in the signing process. Please refer to the document shared on QR.

Document Name: QR Code Format & Structure

Appendix

Openssl commands & urls that can be useful

Openssl:

- Hash function: openssl dgst -sha256 <xml_file_name>
- Generate private key: openssl ecparam -name secp256k1 -genkey -noout -out PrivateKey.pem
- Generate public key: openssl ec -in PrivateKey.pem -pubout -conv_form compressed -out PublicKey.pem
- Generate csr: openssl req -new -sha256 -key privateKey.pem -extensions v3_req -config config.cnf -out taxpayer.csr

URLs:

- XML Canonical online tool: XML Canonicalizer (soapclient.com)
- XPATHER ONLINE TOOL: http://xpather.com/
- Hashing online tool: https://emn178.github.io/online-tools/sha256.html
- Hex to base 64 online: https://base64.guru/converter/encode/hex
- ENCODER BASE64 online: https://www.base64encode.org/
- ECDSA SIGN online: https://8gwifi.org/ecsignverify.jsp
- CSR and certificate decoder online: https://certlogik.com/decoder/
- TEXT to HEXA online: https://www.online-toolz.com/tools/text-hex-convertor.php
- private key decoder online: http://certificate.fyicenter.com/2145_FYIcenter_Public_Private_Key_Decoder_and_Viewer.html#Result
- TLV QR decoder online: https://emvlab.org/tlvutils/

THANK YOU

فـاتورة
Fatoora

هيئة الزكاة والضريبة والجمارك
Zakat, Tax and Customs Authority